



UNIVERSIDAD DE LAS FUERZAS ARMADAS ESPE

DEPARTAMENTO DE CIENCIAS DE LA COMPUTACIÓN

ASIGNATURA: DESARROLLO DE APLICACIONES MÓVILES

Tarea 1.2: Desarrollo aplicaciones móviles con controles básicos en Flutter

Arquitectura Modelo-Vista-Controlador en Flutter

El documento presenta la resolución de 2 ejercicios algorítmicos aplicados al desarrollo móvil (Sistema de cobros y Nómina de choferes), estructurando el código para separar la lógica de negocio de la interfaz visual mediante el patrón MVC, incluyendo diagramas Nassi-Shneiderman y pseudocódigos.

Autores: Diana Guerra, Alejandro Montufar, Paul Moreno, Leonel Tipán

Fecha: 11 de mayo de 2026

Índice

1. Introducción	2
2. Objetivos	2
3. Marco Teórico	2
3.1. Flutter	2
3.2. Dart	2
3.3. Android Studio	3
3.4. Patrón Modelo-Vista-Controlador (MVC)	3
4. Desarrollo de los Ejercicios	3
4.1. Ejercicio 1: Aplicación 1 - Pagos de servicios básicos (pry_deber2)	3
4.1.1. Pseudocódigo	4
4.1.2. Diagrama Nassi-Shneiderman	5
4.1.3. Código (Lógica Central del Modelo)	6
4.1.4. Ejecución	6
4.2. Ejercicio 2: Aplicación 2 - Factura de pago a choferes (pry_deber2_act2)	8
4.2.1. Pseudocódigo	8
4.2.2. Diagrama Nassi-Shneiderman	9
4.2.3. Código (Lógica Central del Modelo)	10
4.2.4. Ejecución	10
5. Conclusiones	12
6. Referencias	12
7. Anexos	12
7.1. Repositorio de código fuente	12
7.1.1. Enlace principal	12

1. Introducción

El presente documento detalla la resolución técnica de dos casos prácticos enfocados en el desarrollo de aplicaciones móviles multiplataforma utilizando Flutter y el lenguaje Dart. Estos ejercicios, correspondientes al Sistema de Pago de Servicios (`pry_deber2`) y a la Nómina de Choferes (`pry_deber2_act2`), fueron abordados mediante el uso riguroso de la Arquitectura Modelo-Vista-Controlador (MVC) y complementados con principios de *Atomic Design* para lograr una separación clara de responsabilidades y modularidad extrema en la interfaz de usuario. A través del uso previo de diagramas Nassi-Shneiderman y pseudocódigos, se modeló la lógica de negocio antes de su implementación final, garantizando una codificación estructurada, eficiente y fácil de mantener.

2. Objetivos

- **Objetivo General:** Desarrollar aplicaciones móviles en Flutter aplicando el patrón MVC y metodologías de *Atomic Design* para la resolución de problemas lógicos y matemáticos, asegurando un diseño limpio y escalable.
- **Objetivo Específico 1:** Modelar las soluciones algorítmicas de cada ejercicio utilizando pseudocódigo y diagramas Nassi-Shneiderman como paso previo indispensable, para validar la lógica pura independientemente del marco de trabajo.
- **Objetivo Específico 2:** Implementar una estructura modular de componentes de interfaz (átomos, moléculas y organismos) que facilite la reutilización, lectura y escalabilidad del código visual (*widgets*) en Flutter.
- **Objetivo Específico 3:** Gestionar cálculos complejos y estructuras de datos dinámicas (como listas de choferes, recargos condicionales y arreglos iterativos) mediante clases puras encapsuladas en la capa de Modelo, liberando así por completo a la Vista de reglas empresariales.

3. Marco Teórico

3.1. Flutter

Flutter es un framework de desarrollo multiplataforma creado por Google que permite construir interfaces nativas para Android, iOS, web y escritorio desde una sola base de código. Su paradigma principal es la programación declarativa con *widgets*, donde la interfaz se describe como un árbol de componentes reutilizables. Flutter utiliza un motor gráfico propio para renderizado, lo que reduce la dependencia de componentes nativos y mantiene consistencia visual entre plataformas.

3.2. Dart

Dart es el lenguaje de programación usado por Flutter. Combina tipado estático opcional, soporte para programación orientada a objetos y características modernas como `async/await` para la concurrencia. Su compilador permite generar código nativo de alto rendimiento para dispositivos móviles y código JavaScript para la web, lo que habilita el enfoque multiplataforma de Flutter.

3.3. Android Studio

Android Studio es el entorno de desarrollo integrado (IDE) oficial para Android y uno de los mas usados para Flutter. Proporciona herramientas de edicion, depuracion, emuladores y administracion de dependencias. En proyectos Flutter integra la ejecucion en dispositivos, el analisis estatico y el soporte para hot reload, lo que acelera el ciclo de desarrollo.

3.4. Patrón Modelo-Vista-Controlador (MVC)

El patron MVC es una arquitectura de diseno de software que separa los datos de una aplicacion, la interfaz de usuario y la logica de control en tres componentes altamente desacoplados:

- **Modelo:** Maneja la logica de negocio y las reglas de procesamiento de datos independientes de la interfaz. Representa el nucleo del comportamiento de la aplicacion.
- **Vista:** Proporciona la representacion grafica y captura los eventos tactiles del usuario (en Flutter, esto se realiza mediante el arbol de *widgets* y *Scaffolds*).
- **Controlador:** Recibe las interacciones del usuario desde la vista, las transforma en peticiones o ejecuciones para el modelo, y devuelve los resultados limpios para actualizar el estado de la vista.

4. Desarrollo de los Ejercicios

4.1. Ejercicio 1: Aplicación 1 - Pagos de servicios básicos (pry_deber2)

Descripción:

Crear una aplicación móvil que permita registrar y calcular pagos de servicios básicos o frecuentes.

Servicios considerados:

- Agua potable
- Energía eléctrica
- Internet y telefonía
- TV por cable y streaming
- Otros pagos frecuentes

Reglas de negocio:

La aplicación debe permitir:

- Registrar datos del cliente.
- Seleccionar el tipo de servicio.
- Ingresar el consumo o valor base del servicio.
- Aplicar una tarifa según el servicio seleccionado.
- Calcular subtotal.

- Aplicar descuentos o recargos según condiciones definidas.
- Calcular el total a pagar.
- Generar un resumen del pago realizado.

4.1.1. Pseudocódigo

Inicio

```

Leer tipoServicio, consumoBase, formaPago
Leer aplicarMantenimiento, aplicarSeguro, aplicarDescuento

tarifa = 1.0
Si tipoServicio == "Agua" Entonces tarifa = 0.50
Si tipoServicio == "Energía eléctrica" Entonces tarifa = 0.15

subtotal = consumoBase * tarifa

descuento = 0
Si aplicarDescuento == Verdadero Entonces
    descuento = subtotal * 0.50
FinSi

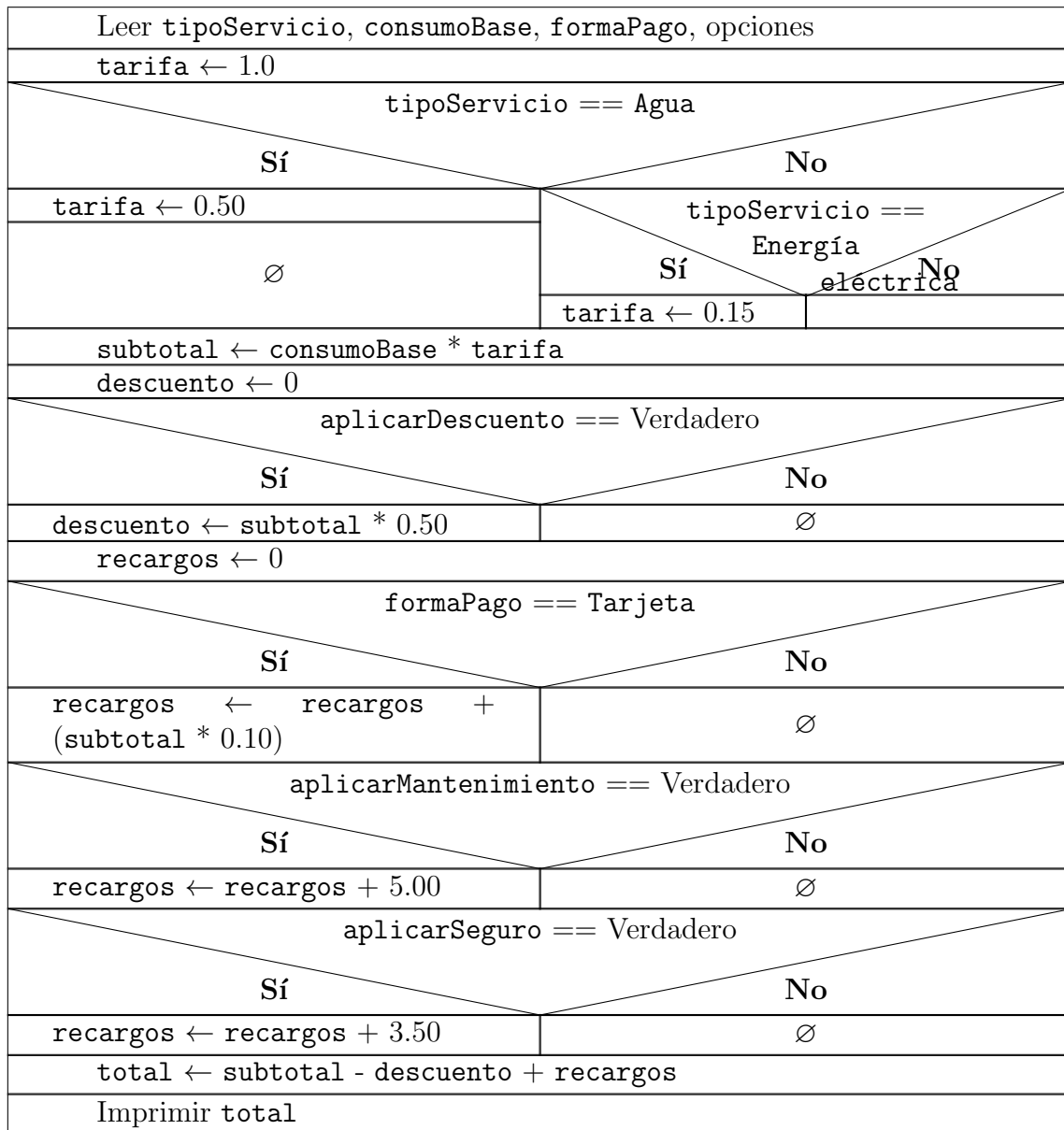
recargos = 0
Si formaPago == "Tarjeta" Entonces
    recargos = recargos + (subtotal * 0.10)
FinSi
Si aplicarMantenimiento == Verdadero Entonces
    recargos = recargos + 5.00
FinSi
Si aplicarSeguro == Verdadero Entonces
    recargos = recargos + 3.50
FinSi

total = subtotal - descuento + recargos
Imprimir subtotal, descuento, recargos, total
Fin

```

El pseudocódigo anterior demuestra un flujo secuencial donde se establecen primero las tarifas base según el servicio seleccionado. Luego, a través de estructuras condicionales simples, se evalúan los diferentes posibles incrementos (recargos por tarjeta, seguro, mantenimiento) y decrementos (descuento) antes de llegar a la totalización del rubro final a pagar.

4.1.2. Diagrama Nassi-Shneiderman



En este diagrama se visualiza de forma mucho más gráfica y estructurada el comportamiento algorítmico. Las sentencias condicionales se representan en bloques que dividen claramente el flujo de datos, ilustrando cómo los cálculos son acumulativos a la variable de recargos y que su ejecución depende estrictamente de las banderas booleanas determinadas por la selección del usuario.

4.1.3. Código (Lógica Central del Modelo)

```
lib/model/pago_model.dart

double calcularSubtotal() {
  double tarifaMultiplicador = 1.0;
  if (tipoServicio == 'Agua') tarifaMultiplicador = 0.50;
  if (tipoServicio == 'Energía eléctrica') tarifaMultiplicador = 0.15;
  return consumoBase * tarifaMultiplicador;
}

double calcularDescuentos() {
  if (aplicarDescuento) {
    return calcularSubtotal() * 0.50;
  }
  return 0.0;
}

double calcularRecargos() {
  double recargos = 0;
  if (formaPago == 'Tarjeta') {
    recargos += calcularSubtotal() * 0.10;
  }
  if (aplicarMantenimiento) recargos += 5.00;
  if (aplicarSeguro) recargos += 3.50;
  return recargos;
}

double calcularTotal() {
  return calcularSubtotal() - calcularDescuentos() + calcularRecargos();
}
```

Como se observa en el bloque de código, la clase `PagoModel` centraliza por completo las reglas de negocio del problema, exponiendo métodos públicos como `calcularTotal()` que trabajan unificadamente usando los atributos instanciados. Al no depender de ningún contexto o widget de Flutter, esta lógica financiera es fácilmente testeable y adaptable en cualquier otra vista o controlador sin sufrir modificaciones.

4.1.4. Ejecución

A continuación, se ilustra la aplicación en pleno funcionamiento. La interfaz refleja un sistema de cartas que encapsulan los controles del usuario, construidas bajo la abstracción de organismos (grupos de inputs) que interactúan asincrónicamente con el modelo descrito arriba.

12:31

Registro de Consumo

Nombre del Cliente

Leonel

Tipo de Servicio

☒ Agua ☐ Energía eléctrica ☐ Internet ☐ Otros

Consumo o Valor Base (\$ / unid)

12

de Pago

☒ Efectivo ☐ Tarjeta

Descuentos:

☒ Tercera Edad / Discapacidad (-50%)

Servicios Adicionales (Recargos):

☒ Mantenimiento Preventivo (+\$5)

☐ Seguro Antidaños (+\$3.50)

Limpiar Calcular

4.2. Ejercicio 2: Aplicación 2 - Factura de pago a choferes (pry_deber2_act2)

Descripción:

Crear una aplicación móvil para calcular la nómina semanal de una compañía de transporte que cuenta con cinco choferes.

Datos requeridos:

De cada chofer se debe registrar:

- Nombre.
- Horas trabajadas por día durante seis días.
- Sueldo por hora.

Reglas de negocio:

La aplicación debe calcular:

- Total de horas trabajadas a la semana por cada chofer.
- Sueldo semanal de cada chofer.
- Total general que pagará la empresa.
- Nombre del chofer que trabajó más horas el día lunes.
- Reporte general con todos los datos calculados.

4.2.1. Pseudocódigo

Inicio

```
Leer N // Número de choferes
totalEmpresa = 0
maxHorasLunes = -1
choferDestacado = ""
```

```
Para i desde 1 hasta N Hacer
```

```
    Leer nombre, sueldoPorHora, recibeBono, horasDiarias[1..6]
```

```
    totalHorasChofer = 0
```

```
    Para j desde 1 hasta 6 Hacer
```

```
        totalHorasChofer = totalHorasChofer + horasDiarias[j]
```

```
    FinPara
```

```
    sueldoSemanal = totalHorasChofer * sueldoPorHora
```

```
    Si recibeBono == Verdadero Entonces
```

```
        sueldoSemanal = sueldoSemanal + 50.0
```

```
    FinSi
```

```
totalEmpresa = totalEmpresa + sueldoSemanal
```

```
Si horasDiarias[1] > maxHorasLunes Entonces
```

```
    maxHorasLunes = horasDiarias[1]
```

```

    choferDestacado = nombre
FinSi

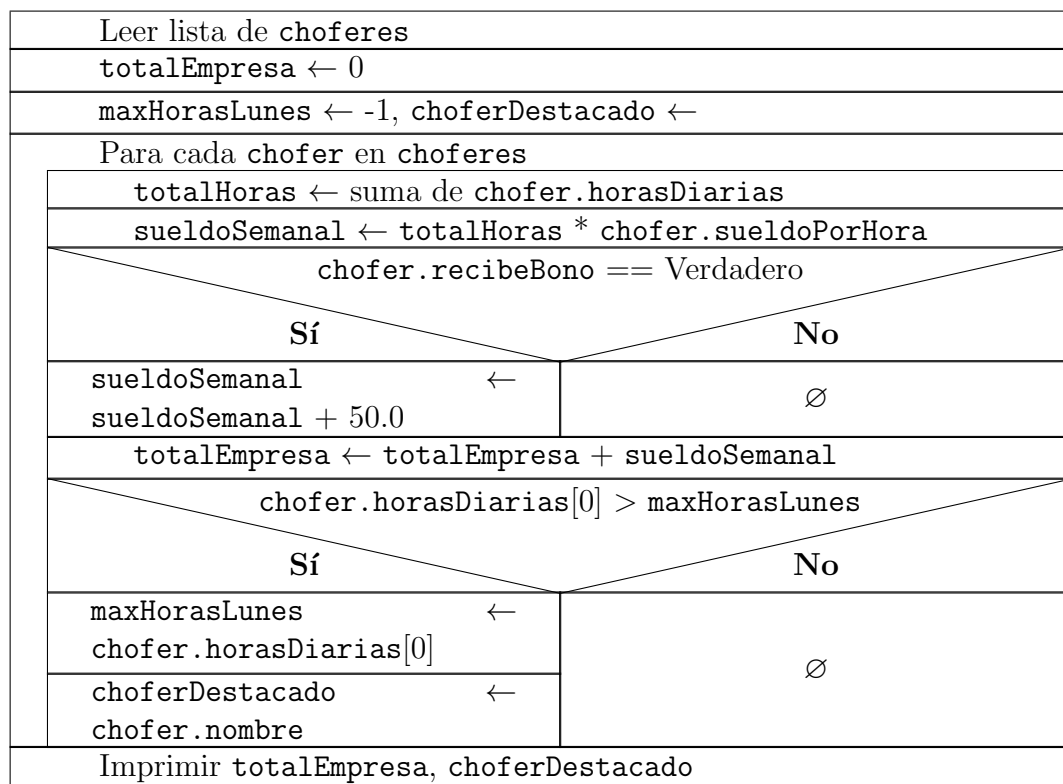
    Imprimir "Chofer:", nombre, "Sueldo:", sueldoSemanal
FinPara

    Imprimir "Total a pagar empresa:", totalEmpresa
    Imprimir "Chofer destacado Lunes:", choferDestacado
Fin

```

Este pseudocódigo describe una lógica de procesamiento mucho más interactiva y de acumulación, donde intervienen arreglos para representar las horas diarias de una jornada semanal de 6 días, además de bucles iterativos que procesan conductor por conductor. Simultáneamente a los cálculos de nómina, se actualiza un registro comparativo para determinar en tiempo real el conductor más activo del primer día (Lunes).

4.2.2. Diagrama Nassi-Shneiderman



El diagrama ilustra el recorrido de la estructura de datos que almacena a los choferes registrados. El bloque cíclico principal encapsula tanto la determinación del sueldo base y la suma del bono especial para cada empleado de forma individual, como también el algoritmo de búsqueda de un máximo, crucial para reportar la eficiencia productiva de la plantilla laboral.

4.2.3. Código (Lógica Central del Modelo)

```
lib/model/chofer_model.dart

class Chofer {
  // Atributos: nombre, horasDiarias, sueldoPorHora, etc.

  double calcularTotalHoras() => horasDiarias.fold(0, (a, b) => a + b);

  double calcularSueldoSemanal() {
    double base = calcularTotalHoras() * sueldoPorHora;
    return recibeBono ? base + 50.0 : base;
  }
}

class NominaModel {
  final List<Chofer> choferes;
  NominaModel({required this.choferes});

  double calcularTotalEmpresa() {
    return choferes.fold(0, (total, chofer) =>
      total + chofer.calcularSueldoSemanal());
  }

  String obtenerChoferMasHorasLunes() {
    if (choferes.isEmpty) return "No se registraron choferes";
    Chofer destacado = choferes.first;
    for (var chofer in choferes) {
      if (chofer.horasDiarias[0] > destacado.horasDiarias[0]) {
        destacado = chofer;
      }
    }
    return destacado.nombre;
  }
}
```

En la implementación final de Dart, podemos resaltar el uso de las funciones de orden superior y métodos declarativos para iterables (como el método `fold` aplicado sobre la lista `horasDiarias` o sobre los propios `choferes`), lo que nos permite contraer múltiples ciclos for convencionales en elegantes líneas compactas. Asimismo, la clase padre `NominaModel` orquesta el cálculo general administrando internamente múltiples instancias de un submodelo base llamado `Chofer`.

4.2.4. Ejecución

La siguiente captura evidencia el diseño moderno e iterativo construido en la interfaz gráfica, en donde las tarjetas dinámicas que representan a cada conductor son un claro ejemplo del uso de "moléculas" atómicas inyectadas directamente dentro de una "plantilla" o "página" mayor, logrando una lista de componentes fluidos y reutilizables.

12:26

← Registro de Choferes

Nombre del Chofer

Diana

Sueldo por Hora (\$)

12

Horas Trabajadas:

Lun	Mar	Mié
5	6	7
Jue	Vie	Sáb
	8	7

Tipo de Jornada

☒ Diurna ☐ Nocturna ☐ Mixta

Estados de Empleado:

☒ Chofer Activo

☐ Recibe Bono Semanal

Registrar Ver Nómina

5. Conclusiones

- La abstracción de algoritmos mediante pseudocódigos y diagramas Nassi-Shneiderman ha permitido diseñar lógicas estructuradas mucho antes de escribir el código fuente, lo que previene errores semánticos y refactorizaciones durante el desarrollo.
- La adopción del patrón MVC en el entorno de desarrollo Flutter ha demostrado ser altamente eficiente para separar la lógica de negocio de la interfaz de usuario. Además, el diseño visual se construyó de manera mucho más modular, aplicando los principios del Diseño Atómico (Atomic Design), separando los *widgets* en átomos, moléculas y organismos, lo que permite una mayor reutilización de código y escalabilidad en los proyectos.
- En la resolución de problemas (como el cálculo de facturas y la gestión de nóminas), encapsular la lógica pura dentro del *Modelo* permitió simplificar el código, haciendo que los controladores manejen validaciones de vista y deleguen el comportamiento matemático sin interrupciones.

6. Referencias

- Chapin, N. (1974). *New Format for Flowcharts*. Software: Practice and Experience, 4(4), 341-357.
- Google (2024). *Flutter Documentation*. Recuperado de <https://docs.flutter.dev/>
- Google (2024). *Dart Language Overview*. Recuperado de <https://dart.dev/overview>
- Google (2024). *Android Studio User Guide*. Recuperado de <https://developer.android.com/studio/intro>
- Gamma, E., Helm, R., Johnson, R., y Vlissides, J. (1994). *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley.
- Oracle (2024). *Java Platform, Standard Edition: Java Language Specification*. Recuperado de <https://docs.oracle.com/javase/specs/>

7. Anexos

7.1. Repositorio de código fuente

7.1.1. Enlace principal

- https://github.com/LeonelTQL/moviles/tree/main/pry_deber2
- https://github.com/LeonelTQL/moviles/tree/main/pry_deber2_act2